This script is designed for a team of four presenters (Helal, Saraf, Zaman, and Habiba) to present the 10-slide deck on the **Avoid-3 (Reverse Tic-Tac-Toe)** project. The language is kept simple for clarity.

Slide 1: Title Slide (Introduction)

Speaker 1 (Helal): Hello everyone. Today, our team is excited to present our Artificial Intelligence project called **"Avoid-3."** This is a unique version of Tic-Tac-Toe where the goal is actually to avoid getting three in a row. I am Helal, and I am joined by my teammates Saraf, Zaman, and Habiba.

Slide 2: Problem Statement

Speaker 1 (Helal): Standard Tic-Tac-Toe on a 3x3 board is too easy for modern computers because they can calculate every possible move. To make it harder, we moved to a **4x4 grid**. This larger board has over **16 trillion** possible move sequences, making it much more complex for an AI to solve without smart algorithms.

Slide 3: Objective

Speaker 2 (Saraf): Our main goal was to implement **Adversarial Search** in a "Misère" context. In simple English, "Misère" means the rules are inverted: if you complete a line of three, you **lose immediately**. We wanted to build an AI that could play this game rationally and beat human players.

Slide 4: Tools & Technologies

Speaker 2 (Saraf): We used **Python 3** to write our code because it is excellent for AI development. We also used the standard `math` **library** for our calculations. The game runs in a simple console window where players enter numbers from 1 to 16 to pick a square.

Slide 5: Implementation – Board Design

Speaker 3 (Zaman): We represented the 4x4 board as a **Python list** of 16 spaces. To check for losses, we created a list called `LOSS_LINES`. This contains every possible row, column, and diagonal of three. The computer constantly checks if any player has accidentally filled one of these lines.

Slide 6: Implementation – AI Core

Speaker 3 (Zaman): The AI uses the **Minimax algorithm** to make decisions. Because the 4x4 board is so large, the AI cannot see the very end of every game. Instead, we set a **depth limit of 6**, meaning the AI looks ahead six turns to find the best move.

Slide 7: Implementation – Optimization

Speaker 4 (Habiba): To make the AI faster, we added **Alpha-Beta Pruning**. This technique allows the AI to "prune" or skip branches of the game tree that it knows are bad. This effectively doubles the AI's searching speed, allowing it to respond to human moves in real-time.

Slide 8: Implementation – Heuristic

Speaker 4 (Habiba): Since the AI doesn't always see the end of the game, it uses a **Heuristic Scoring** function to guess who is winning. It gives points for having more **unmatched tiles**—which are tiles not part of any line—and takes away points if the AI moves into a "danger zone".

Slide 9: Limitations

Speaker 1 (Helal): Our project does have some limits. Because of the **6-ply depth constraint**, the AI might sometimes miss a very clever trap set by a human that takes more than six moves to complete. Also, our current heuristic is an approximation and isn't a "perfect" solution to the game.

Slide 10: Further Work & Thank You

Speaker 1 (Helal): In the future, we hope to add a **Graphical User Interface (GUI)** to make the game look better. We also want to train a **Neural Network** to help the AI make even smarter guesses.

Speaker 1 (Helal): Thank you for listening to our presentation. We hope you enjoyed learning about Avoid-3. Does anyone have any questions?

Would you like me to prepare a practice quiz for your team to test your knowledge before the actual presentation?